



Dakar Institute of Technology Data Engineering, Spark et Scala

Système d'analyse de données e-commerce distribué

De quatre fichiers bruts aux indicateurs métier

Groupe 7

DJIDOHOKPIN Samuel BALDÉ Azizatou DIALLO Cheick Oumar

Soutenance le 5 septembre 2026

Plan

- 1 Le problème
- 2 Notre solution
- 3 Les trois modules
- 4 Démonstration

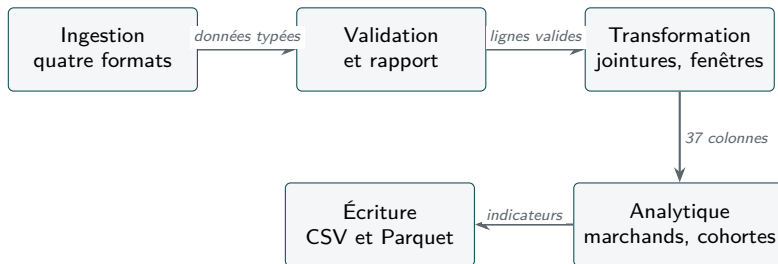
Le problème posé

Quatre fichiers, quatre formats, aucune garantie sur leur contenu. Il faut en tirer des indicateurs sur lesquels une équipe métier puisse décider.

Source	Format	Volume
transactions	CSV	138 047 lignes
users	JSON, un objet par ligne, un champ imbriqué	12 000 lignes
products	Parquet, réparti sur 12 fichiers	6 000 lignes
merchants	CSV	600 lignes

Les données sont volontairement sales. Le sujet nous prévient : un taux de rejet nul signifierait une erreur d'implémentation.

L'architecture du pipeline



Une application unique orchestre la chaîne et chronomètre chaque étape.

Chaque étape peut aussi être lancée seule, ce qui a permis à chacun de tester sa partie sans attendre celle des autres.

L'organisation du groupe

Membre	Périmètre	Fichiers dont il est propriétaire
Samuel	ingestion et plateforme	modèles, lecture, validation, configuration
Azizatou	transformation	fonction temporelle, enrichissement, fenêtrage
Cheick Oumar	analytique et performance	indicateurs, optimisations, application

Une branche par membre, et un découpage tel que personne ne modifie le fichier d'un autre.

Treize fichiers Scala, aucun conflit de fusion sur le code. Le seul heurt a porté sur le journal de contribution, le seul fichier que nous éditons tous les trois.

Chaque module est relu par un membre qui ne l'a pas écrit. **Cette relecture a rattrapé une erreur qui empêchait tout le projet de compiler** : une valeur déclarée immuable, puis réaffectée.

Les choix techniques structurants

Scala 2.12, Spark 3.3, Java 11

La contrainte vient de la plateforme : Spark 3.3 impose Scala 2.12 et refuse de tourner sous Java 17. Nous ne les avons pas choisies, nous les avons constatées puis assumées.

Des jointures à gauche depuis les transactions

Les données contiennent des références vers des clients, des produits ou des marchands qui n'existent pas. Une jointure interne les supprimerait, et le chiffre d'affaires en serait faussé. Concrètement, 243 identifiants de marchands cités par des transactions sont absents du référentiel : ils pèsent 3,1 pour cent du chiffre d'affaires, que nous conservons.

Aucun paramètre écrit en dur

Chemins, seuils de validation, réglages Spark, activation du cache : tout est dans un fichier de configuration, et chaque clé absente a une valeur de repli.

Mon module

Lire quatre formats avec quatre stratégies distinctes, typer le résultat, puis séparer les lignes exploitables des lignes douteuses en conservant le motif de chaque rejet.

2 428

lignes écartées au total, sur les quatre jeux de données

Ma difficulté

Une comparaison portant sur une valeur absente ne renvoie pas faux, elle renvoie une valeur absente. Sans précaution, les lignes vides passaient au travers du filtre au lieu d'être écartées.

aucune

ligne perdue : chaque rejet est conservé avec son motif

Mon module

Extraire six caractéristiques temporelles de chaque horodatage, rattacher les trois référentiels aux transactions, puis mesurer le comportement des clients sur des fenêtres glissantes.

136 157

transactions enrichies, aucune perdue en chemin

Ma difficulté

Repérer les clients actifs au moins cinq jours distincts sur sept. Spark ne sait pas compter des valeurs distinctes dans une fenêtre : il a fallu reformuler le problème plutôt que forcer la formule.

9 puis 37

colonnes : ce que l'enrichissement ajoute à une transaction

Mon module

Produire les indicateurs par marchand et l'analyse de fidélité par cohorte, régler les optimisations de Spark, et orchestrer l'ensemble dans une application unique.

843

lignes dans le rapport marchands : 586 identifiés,
257 orphelins

Ma difficulté

Spark recalculait tout le pipeline à chaque résultat demandé. Il fallait décider quoi conserver : le tableau enrichi, trop volumineux pour la seule mémoire, est persisté sur disque ; la matrice de cohortes, plus petite, est simplement mise en cache.

2 ans

d'historique découpé en cohortes mensuelles

Démonstration

Le pipeline complet, puis la lecture des résultats métier

Merci de votre attention

DJIDOHOKPIN Samuel BALDÉ Azizatou DIALLO Cheick Oumar
Groupe 7, Dakar Institute of Technology

Vos questions

Réserve : le rapport de qualité des données

Jeu de données	Lues	Valides	Rejetées	Taux	Valeurs nulles
transactions	138 047	136 157	1 890	1,37 %	940
users	12 000	11 655	345	2,88 %	160
products	6 000	5 821	179	2,98 %	55
merchants	600	586	14	2,33 %	9

Ce rapport est lui-même un livrable : il est affiché en console et écrit sur disque, au même titre que les indicateurs métier.

Réserve : lancer le projet

Voie	Usage	Résultat
SBT	développement et assemblage	archive de 5,8 Mo
scala-cli	exécution sans installer SBT	archive de 367 Ko
spark-submit	déploiement sur un cluster	consomme l'archive

L'archive livrée n'embarque pas Spark, qui est fourni par le cluster.

C'est le principe d'une application mobile, qui n'embarque pas le système d'exploitation du téléphone.

L'application accepte aussi un argument d'étape : ingestion, transformation, analytique, ou le pipeline entier.

Réserve : ce que nous n'avons pas démontré

- Le pipeline n'a jamais tourné sur un vrai cluster. Tout a été exécuté en local, donc le passage à l'échelle reste une hypothèse.
- Aucun test unitaire automatisé n'accompagne le code, alors que la bibliothèque de test est déclarée dans le projet.
- Le gain réel des optimisations n'est pas chiffré : nous savons qu'elles évitent des recalculs, nous n'avons pas mesuré combien.
- Les données nous ont été fournies. Elles ne viennent pas d'un système en production, avec ce que cela suppose de surprises en moins.